Week 3: Representation – Undirected Graphs
Practical Assignment

For this week's practical, we look at the following two problems:

- The Hamming (7,4) error-correction code, for which we set up the model as Bayes network (BN), and then convert it to a Markov network (MN) and a factor graph (FG).
- Image denoising for a binary image, for which we set up the model as a MN from the start.

For both problems we still follow a very simple inference approach: We first construct the joint distribution over all the RVs in the model, and from this joint distribution calculate the marginal belief distributions over the latent (hidden/unobserved) RVs we are interested in. For the second problem, we will encounter the limitations of this simple inference approach. We will revisit both of these problems later in the module and implement more efficient inference algorithms.

1 The Hamming (7,4) error-correction code

The Hamming (7,4) error-correction code is applied to digital information transmitted over a noisy communications channel. This simple code extends a 4-bit input sequence with a further 3 parity check bits to result in a 7-bit sequence to transmit. This provides redundancy that allows automatic correction of any *one* wrongly received bit. It works as follows:

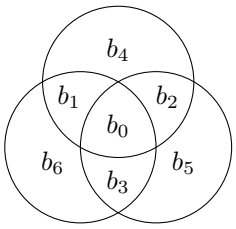


Figure 1: Creating parity checks in the Hamming (7,4) error correcting code. Bits $b_0, \ldots, b_3$ are the original four data bits to be transmitted. Bits b_4, b_5 and b_6 are chosen to create even parity in each of the three circles they occur in.

Transmitter side: Figure 1 describes the coding of the original data to be transmitted. b_0, b_1, b_2 and b_3 are the original 4 information bits to be transmitted, while b_4, b_5 and b_6 are so-called parity check bits. These are chosen so that there will be an even number of 1's in each of the three circles – so-called 'even' parity.

Receiver side: The decoder will receive seven bits, $r_0, \ldots, r_6$. Let us consider the three corresponding circles from Figure 1, but now with $r_0, \ldots, r_6$:

- If all three circles have even parity, the most likely conclusion is that $r_0, \ldots, r_6$ were decoded (recreated) correctly. The alternative would be that there is an even number of errors in each of the three circles, which common sense should indicate as a much less likely outcome. (You will soon be able to calculate these probabilities.)
- If exactly one of these circles fails its parity check, the most likely conclusion is that the bit unique to that circle – i.e., the parity bit – is the one that is in error.

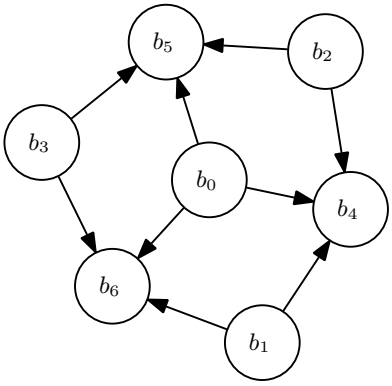
- If exactly two of these circles fail their parity checks, the most likely conclusion is that the single bit that is common to both these circles, but not also common to the third circle, is the one that is in error.
- If all three of them fail their parity checks, the most likely conclusion is that the bit common to them all – i.e. r_0 – is in error.
- Note that if there are more than one error, the decoding will give an incorrect answer. This code can only correct one error in the seven transmitted bits (of which only four are the original information bits we wanted to transmit).

This is enough information to be able to implement such a coder/decoder pair. A few simple rules on the transmit side can generate the check bits for any arbitrary $b_0, \ldots, b_3$. On the receive side, one can calculate the three parities (one for each circle) and then use the logic above to figure out which bit (if any) is in error. However, we will now tackle this via PGMs. Instead of explicitly coding these particular rules, we are going to capture their logic in probability distributions (you can think of this as declarative coding) and then let the PGM figure out what goes for what[1].

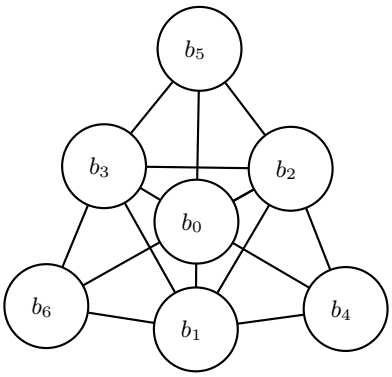
1.1 Model the tranmitter side with data bits $b_0, \ldots, b_3$ and check bits b_4, b_5 and b_6

We start building our model by first focussing on the transmitter side; i.e., we model how the data bits are generated and the check bits are calculated. In the next subsection, we will then extend the model to include the transmission of data over a noisy channel.

(a) (On paper) Your first task is to convert Figure 1 into a BN and make a drawing of it (i.e., capture the relationships between the bits on the transmitter side by using a BN). Try to do this on your own – a primary skill you want to develop is the art of transforming logical constraints into a corresponding probability distribution. You can confirm your answer by looking at Figure 2(a).



(a) Bayes network (BN)



(b) Markov network (MN) (Find the fourth maximal clique!)

Figure 2: BN and MN representations for the Hamming (7,4) coding stage.

(b) (On paper) Directly from this BN, write down a symbolic expression for the joint distribution $p(b_0, \ldots, b_6)$. Specifically note how the joint distribution, representing the whole coding subsystem, factorises into a number of smaller distributions where each of them describes a very specific "local" aspect (or piece of knowledge) pertaining to the system. (You should get: $p(b_0, \ldots, b_6) = p(b_4|b_0, b_1, b_2)p(b_5|b_0, b_2, b_3)p(b_6|b_0, b_1, b_3)p(b_0)p(b_1)p(b_2)p(b_3)$.)

Note the equivalence of two viewpoints on the task – initially we described the procedure from a logical viewpoint. Now we have compiled a set of probability distributions that captures that same logic.

(c) (On paper) Now you need to flesh out the actual probability tables for all those conditional probability distributions that you found via the BN. For instance consider $p(b_4|b_0, b_1, b_2)$: For this you need to think

[1] If the setup and goal is not yet clear to you, think of the problem in this way: We are "at" the receiver side, we receive the 7 bits, $r_0, \ldots, r_6$, and we want to determine what the 4 transmitted data bits, $b_0, \ldots, b_3$, were.

carefully about which combinations of b_0, b_1, b_2 and b_4 have even parity and assign a probability of 1.0 to those cases. All the uneven parity cases get a probability of zero. You should end up with half of the 16 cases having a probability of one, and the other half with a probability of zero.

The distributions for the other parity check factors are similar to this, you only need to substitute to the new sets of random variables.

(d) (On paper) From this, use the moralisation procedure to transform this BN into an Markov network (MN). Your MN should look something like Figure 2(b).

Notice that the conditional distributions of the BN morphs into "cliques" in the MN. They are somewhat more difficult to spot now. You should be able to easily identify the three parity bit cliques. However, there is a fourth maximal clique in this diagram, see if you can spot that – it will surface again later in this course.

(e) (On paper) In the next assignment we will introduce the concept of message passing or belief propagation. In doing this we will almost exclusively focus on two other PGM representations: the factor graph (FG) and the cluster graph (CG).

In preparation for the next assignment, sketch the FG version of the above graphs. The FG makes the relationship between the factors (probability distributions here) and the random variables they operate on, explicit. To sketch this you list your random variables (each shown inside a circle), and then connect them to the factors they appear in (each shown inside a box). Figure 3 shows this.

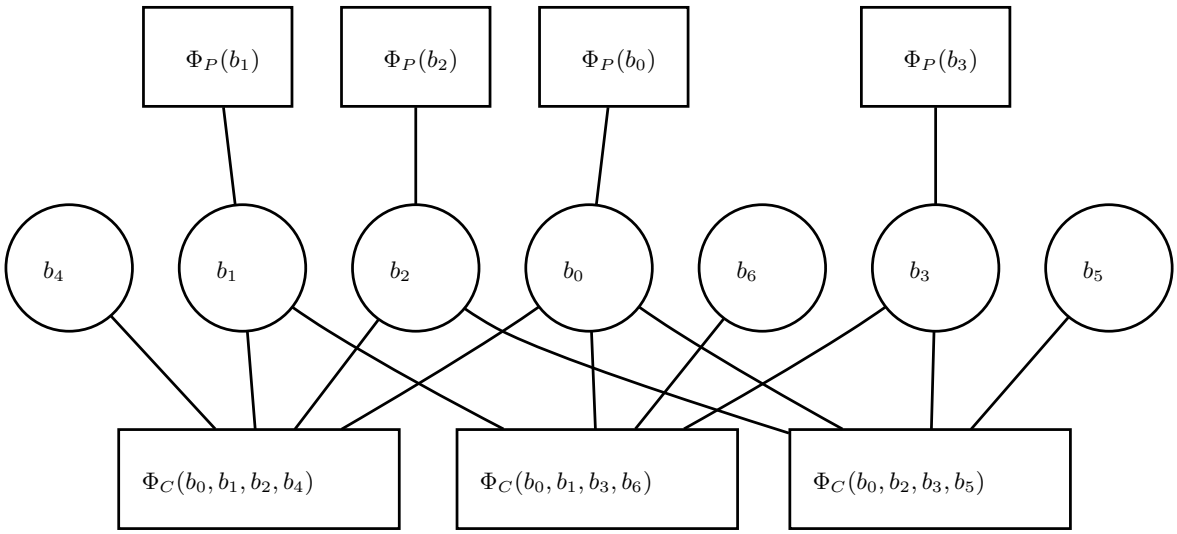


Figure 3: Factor graph (FG) representation for the Hamming (7,4) coding stage.

(f) (In code) Use the `emdw DiscreteTable` class to implement the three factors – one for each parity check circle.

(g) (In code) Multiply these three[2] distributions to get the distribution jointly describing $b_0, \ldots, b_6$, using the `absorb` operator in `emdw`). Use the `normalize` operator to ensure that the result still sums to one.

(h) (In code) Choose some arbitrary combination for the input bits, such as $b_0 = 1, b_1 = 0, b_2 = 1, b_3 = 0$. Use these values as observation or evidence values; do this with `observeAndReduce` in `emdw`. This should result in a single/unique combination of $b_4, \ldots, b_6$ with a non-zero probability (unity after normalisation). You can confirm this by simply printing this (reduced) distribution to the screen. This essentially generates the appropriate values for the check bits given the original four information bits (check that this is in fact correct according to the problem statement). Congratulations, you have built the coding stage for the Hamming(7,4) code.

[2]From the BN, you might have noticed that there also are four other factors describing for $b_0, \ldots, b_3$ their prior probability of being either a one or zero. In general we need to include them. However, if these prior probabilities are "flat" – i.e., equally likely for both cases – they wont affect anything. If you still feel uneasy about leaving them out, feel free to include them. In that case they will also have to be included in the product to determine the joint distribution.

Ponder: In the model of the transmitter we just built, we assumed no knowledge about the sequence of data bits (or – equivalently – we assumed that all combinations are equally likely). How would you adapt your model if you *do* have knowledge that certain combination of data bits are more/less likely?

1.2 Expand the model to include the received bits $r_0, \ldots, r_6$

On the receiving side, the demodulator will receive degraded versions of the transmitted waveforms and apply some decision circuitry to directly translate it into a received binary sequence $r_0, \ldots, r_6$[3]. The error correction logic is not involved in determining this initial received sequence – each received r_i depends directly and solely on what the corresponding transmitted b_i was. Our purpose now is to take a look at these received r_i bits, take into account the legitimate combinations implied by the check-bit coding, and from all of this determine what the most likely transmitted sequence $b_0, \ldots, b_6$ is. Lets approach this systematically:

(a) (On paper) Our first objective is to determine the full BN that describes the receiver side – it therefore should describe the joint distribution for *both* the b_i's as well as the r_i's. Before reading further, first give it a try right now. If you do get stuck, here is some help:

- At the receiver side our knowledge about the parity behaviour between the various b_i's remains unchanged. That part of the network therefore should also remain unchanged, but with the (important) difference that we now of course do not know what the values of the b_i's are (they remain unobserved or latent).
- Each r_i causally depends on its corresponding b_i. We represent this new knowledge by adding an arrow from each b_i to its corresponding r_i.

  Ponder: We know from probability theory that we should not confuse correlation with causality. In principle, we should also have been able to somehow work with $p(b_i|r_i)$. Why did we specifically choose the causal direction?

Easy isn't it? We simply told the system of the applicable facts. Your BN should look something like Figure 4.

(b) (On paper) Directly from this BN, write down an expression for the joint distribution $p(b_0, \ldots, b_6, r_0, \ldots, r_6)$. You should see that the bits that describe the transmitted check bits are exactly as they were on the transmitted side. This makes sense, our knowledge about that hasn't changed. However, there now appears seven $p(r_i|b_i)$ factors.

(c) (On paper) You already have the probability tables for the parity checks. Now compile a table that describes the relationship between r_i and b_i, or $p(r_i|b_i)$. Choose an error probability P_e giving the chances for r_i to differ from the transmitted b_i. Low values implies a good channel, high values a bad channel. Initially you can set $P_e = 0.1$.

(d) (On paper) Drawing the MN for the receiving side should now also be quite easy. Seeing that each r_i only has a single parent, no further moralisation is required. It should look something like Figure 5. If you wish to you can also similarly create an FG version.

(e) (In code) Use the `DiscreteTable` class and to your existing parity check factors, add seven more for the $p(r_i|b_i)$'s.

(f) (In code) Simulate observed values for these r_i's by setting them to the same values as their corresponding b_i's (which of course the receiver is ignorant about), and then flip the value of one r_i to create a single error in those seven received bits. Reduce the various $p(r_i|b_i)$'s with these observed values.

Ponder: Notice that we are doing the observing on the left side of the conditioning bar (i.e., the | symbol). This is quite handy – in your previous probability theory education you went through a bit of Bayes rule gymnastics to achieve this. Another thing you might notice: we now reduced these factors fairly early on, while earlier (with the coding side of things) we did it only after we multiplied the factors. Both are valid (here we wanted to keep the tables small).

[3] If this does not make sense to you, just assume that the received message will be a sequence of bits, $r_0, \ldots, r_6$, which will be the same as the sequence of transmitted bits, $b_0, \ldots, b_6$, but some bits may have "flipped" in the process.

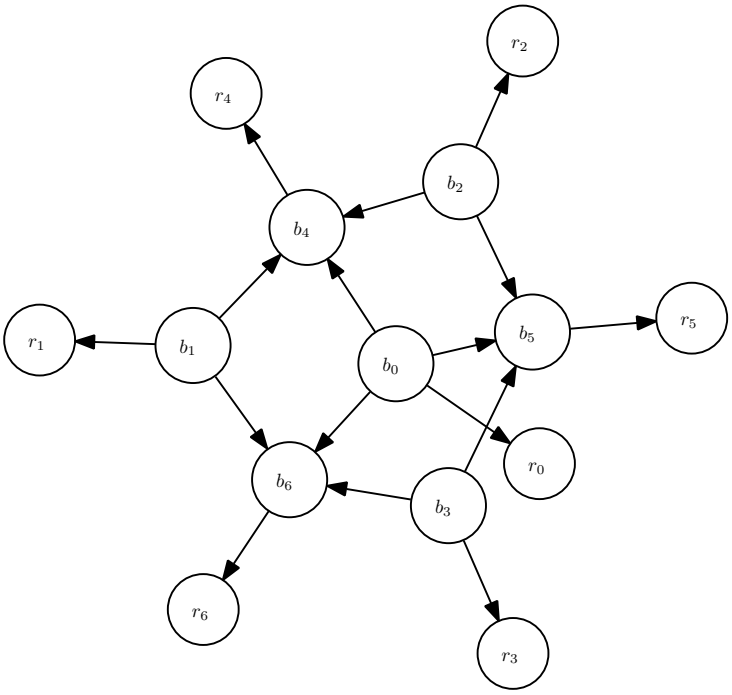


Figure 4: BN representation for the Hamming (7,4) decoding stage.

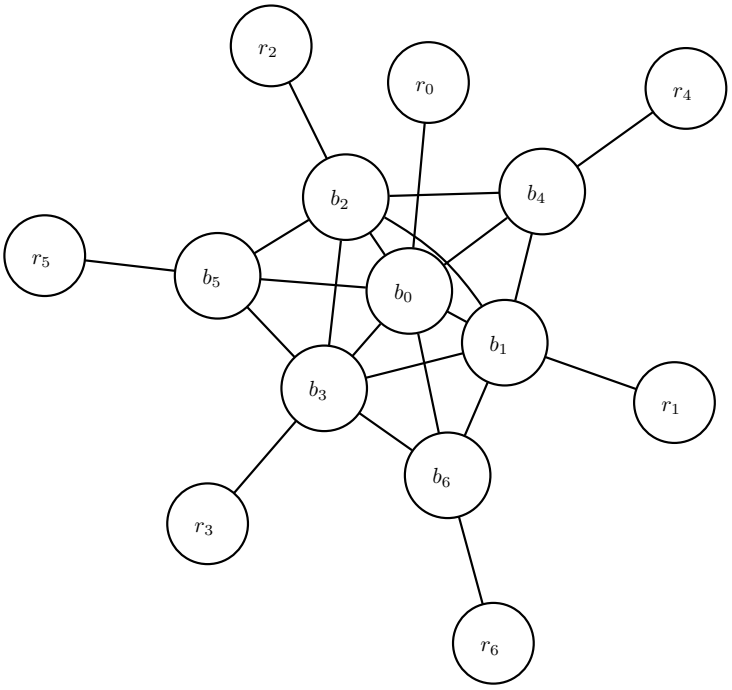


Figure 5: MN representation for the Hamming (7,4) decoding stage.

(g) (In code) Now multiply all your factors together and normalise. Because this product already takes the observed r_i's into account, you have now calculated $p(b_0, \ldots, b_6 | r_0, \ldots, r_6 = \text{the observed values})$. Find the combination with maximum probability, that should be your decoded sequence. You should see that the introduced error was automatically corrected.

Also calculate and display the posterior marginal beliefs for each of the bits, i.e., $p(b_0 = 1|\text{observed values})$, $p(b_1 = 1|\text{observed values})$, etc.[4] (This will be useful to compare to the results of future practicals, where the full joint posterior belief will not be available.)

Ponder: In contrast to the transmitter side of things where there was only one non-zero probability, there now are 16 cases with non-zero probability. Why is this so?

(h) (In code) Play a bit with this. For instance, change P_e and observe what happens to the decoding probabilities. Check whether the maximum probability depends on which particular bit went bad. Flip a second one of the received bits and confirm that this system cannot correct two wrongly received bits. And reflect a moment on how much easier this is than doing the manual calculation.

2 Image denoising

This question is based on **Barber** Example 4.2 (p. 72) and **Bishop** Section 8.3.3. Study these sections in the resources if you have not done so already.

This problem is actually quite similar to Question 1: There is a binary image that is unobserved or latent (similar to the transmitted bits in Question 1), of which a number of pixels are corrupted by randomly flipping their values – this corrupted image is observed (similar to the received bits in Question 1). The goal of the problem is to "clean up" the image by inferring the pixel values of the latent (or original) image x_i given the observed corrupted pixel values y_i.

The model is a pair-wise MN with the following factors:

- A factor over each latent pixel value x_i and its (possibly corrupted) observed value y_i, representing the fact the latent and observed pixel values tend to be similar. The parameter for this factor is called β in **Barber** and η in **Bishop**.
- A factor over each pair of neighbouring latent pixels, x_i and x_j, representing the fact that neighbouring pixels tend to have similar values. Note that by *neighbours*, we mean pixels that are above/below or left/right of each other – pixels that are diagonally across from each other are not considered neighbours. The parameter for this factor is called α in **Barber** and β in **Bishop**.
- In **Bishop** only, there is an additional factor over each latent pixel x_i, representing the fact that pixels tend to prefer one value more than the other. The parameter for this factor is called h. For the first version of your solution, you can omit this factor (which will effectively model that pixels have no preference for either of the two values).

For this practical, we will work with the two binary images that are illustrated in Figure 6: a 4×4 image of which about 10% of the pixels are corrupted, and a 28×28 image of which also about 10% of the pixels are corrupted.

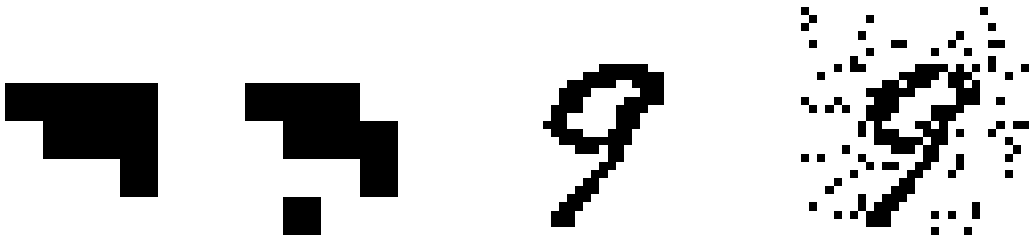


Figure 6: Images used in this question (from left to right): clean 4×4 image `small_img_clean.txt`, corrupted 4×4 image `small_img_noisy.txt`, clean 28×28 image `noisy_img_clean.txt`, corrupted 28×28 image `large_img_noisy.txt`

(a) (On paper) Write the factor over latent pixel x_i and observed pixel y_i as a discrete table in terms of the parameter β or η. Also write the factor over neighbouring pixels, x_i and x_j, as a discrete table in terms of the parameter α or β.

[4] If `ptrB0` points to the `DiscreteTable` object that represents the posterior marginal belief over b_0, then the probability that $b_0 = 1$ can be extracted by using `ptrB0->potentialAt({b0}, {T(1)})`.

(b) (On paper) Choose reasonable values for the parameters to start with. Remember that the factors of a MN can in general not be interpreted as distributions, and it is usually not possible to derive the factor parameters. One usually starts with parameter values that make sense, construct the model, perform inference, and iteratively refine the parameters to get better performance[5]. A simple approach to choose initial parameter values is to look at the ratio of table entries for a factor. For example, for the factor over x_i and y_i, if ratio of corrupted to uncorrupted pixels is 1:10, choose the table entries for $x_i = y_i$ to be 10 times the table entries for $x_i \neq y_i$.

(c) (In code) The file `week03_Q2_framework.cc` provides the framework for this question: It reads the corrupted small image from the file `small_img_noisy.txt` into a matrix, writes this matrix to the screen, illustrates how to define the RVs for the model, creates a matrix that stores the reconstructed image (filled with values of 0.5, for now), and writes this matrix to the text file `small_img_rec.txt`. Add this source file as an application to `emdw`, compile it, and make sure that it executes correctly[6] The file `week03_Q2_framework.cc` contains plenty of comments that explain the code, as well as point out to you where and how you should add code; read through the file and make sure you understand everything. Inspect the contents of `small_img_noisy.txt` and `small_img_rec.txt`. It will be helpful to visualise the clean, corrupted and reconstructed images, using e.g. Python (see visualisation tips further down); I suggest you put this functionality in place.

(d) (In code) For the 4×4 image, construct the factors in `emdw` and multiply them with a running product to form the joint distribution, insert the evidence (the values of the pixels of the corrupted image)[7], and normalise the joint distribution. Note that when you create the factor over the latent x_i and observed y_i for each pixel, it is more efficient to first insert the evidence into the factor *before* you multiply this factor with the running product, instead of first constructing the joint distribution over all the x and y RVs of the model, and then inserting the evidence[8].

(e) (In code) For each pixel, extract the marginal belief $p(x_i = 1|\text{the distorted image})$ from the joint distribution (similar to what you have done in Question 1(g)). Store these beliefs in the matrix that represents the reconstructed image (and which is written out to the file `small_img_rec.txt`). Inspect and visualise the results – can you make sense of it?

(f) (In code) Adjust the two parameters of the model (α or β, and β or η) and observe the effects on the results. Can you explain the change in results? Can you find a set of parameter values such the results are the best (for this image)?

(g) (On paper) Calculate the size of the table necessary to represent the joint distribution for the 4×4 image. Now do the same for the 28×28 image. Can you see why we do not even try to construct the model for the 28×28 image with our current approach to inference? (In the following weeks, we will look at much more efficient inference techniques, and then we will revisit this model.)

Tips for processing and visualisation of data

I use Python to process and visualise data. If you also want to use Python, you might find the following helpful. Most of the following assumes that you have included `numpy` and `matplotlib` as:

```
import numpy as np
import matplotlib.pyplot as plt
```

Reading a reconstructed image from file: If a reconstructed image is stored in the text file `small_img_rec.txt` (containing floating-point values in the interval $[0.0, 1.0]$), then you can load the image as a `numpy` array using:

```
small_img_rec = np.loadtxt('small_img_rec.txt')
```

[5] One can automate this process by learning parameters from data. We will get to this later in the module

[6] Make sure that the file `small_img_noisy.txt` is located in the same directory as that of the compiled application. You might also have to run the application from the directory that contains the compiled application and data file, otherwise the application will not be able to find the data file.

[7] Note that the pixels of the distorted image in the text file can have values of 0 or 1, whereas according to the model as defined in **Barber** and **Bishop**, the pixels can have values of -1 or 1. When you insert the evidence, you therefore have to convert a value of 0 to -1.

[8] To see why, calculate and compare the size of the table for the joint distribution containing both x and y RVs against the size of the table for the joint distribution containing only the x RVs.

Displaying a numpy array as an image: If the 2-D `numpy` array `small_img_rec` contains floating-point values in the interval $[0.0, 1.0]$, it can be displayed as a grayscale image using:

```
plt.figure()
plt.imshow(small_img_rec, cmap='gray_r', vmin=0, vmax=1)
plt.show()
```

3 Assessment: Class test

There will be short, written class test at the end of the practical period in the practical venue (see STEMLearn for details). If you have worked through the practical assignment and understood everything, then the test should be easy!